

```

import datetime

def date_future(nb_jours):
    """Renvoie la date située nb_jours après aujourd'hui"""
    return datetime.date.today() + datetime.timedelta(days=nb_jours)

# Variable contenant les délais en jours pour chaque niveau (index 0 à 4)
DELAIS = [1, 3, 7, 15, 30]

class Carte:
    def __init__(self, question, reponse):
        self.question = question
        self.reponse = reponse
        self.niveau = 0

    # À la création, la carte est à réviser le jour même
    self.date_prochaine = datetime.date.today()

    def __repr__(self):
        return f"""
#####

# Écrire la méthode traiter_reponse(self, succes) de la question 1 #

#####

def traiter_reponse(self, succes):
    if succes:
        self.niveau += 1
    else:
        self.niveau = 0
        delaisProchain = DELAIS[self.niveau]
        self.date_prochaine = date_future(delaisProchain)

# Des cartes et un paquet de cartes pour réaliser des tests
c1 = Carte("Capitale de l'Italie ?", "Rome")
c1.niveau = 2
c1.date_prochaine = date_future(4)

```

```

c2 = Carte("7 x 8 ?", "56")

c2.date_prochaine = date_future(1)

c3 = Carte("Symbole du Fer ?", "Fe")

c3.date_prochaine = date_future(7)

paquet = [c1, c2, c3]

#####

# Écrire la fonction extraire_cartes_du_jour de la question 2 #

#####

def extraire_cartes_du_jour(paquet, date_jour):
    cartesDuJour = []
    for carte in paquet:
        print(carte.date_prochaine)
        if str(carte.date_prochaine)[:3] <= date_jour[:3]: # on vérifie l'année
            if str(carte.date_prochaine)[5:6] <= date_jour[5:6]: # on vérifie le mois
                if str(carte.date_prochaine)[8:] <= date_jour[8:]: # on vérifie le jour
                    cartesDuJour.append(carte)
    return cartesDuJour

#####

# Fonction défaillante à analyser et corriger pour la question 3 #

#####

def extraire_cartes_a_renforcer(paquet):
    """
    Parcourt le paquet et renvoie la liste des cartes ayant le
    niveau d'avancement le plus faible.
    """
    if len(paquet) == 0:
        return []
    niveau_min = paquet[0].niveau
    a_renforcer = []

```

```
for carte in paquet:
    if carte.niveau < niveau_min:
        niveau_min = carte.niveau

a_renforcer.clear() # correction de la fonction extraire_cartes_a_renforcer pour la question 3
a_renforcer.append(carte)

elif carte.niveau == niveau_min:
    a_renforcer.append(carte)

return a_renforcer

def test_reforcement():
    # Création d'un paquet de test

    c1 = Carte("Capitale de l'Italie ?", "Rome")
    c1.niveau = 2

    c2 = Carte("7 x 8 ?", "56")
    c2.niveau = 1

    c3 = Carte("Symbole du Fer ?", "Fe")
    c3.niveau = 2

    mon_paquet = [c1, c2, c3]

    # Appel de la fonction défaillante

    resultat = extraire_cartes_a_renforcer(mon_paquet)

    print("Cartes à renforcer (niveau le plus bas attendu : 1) :")

    print(resultat)
```